High-level Synthesis for Security

Dr. Chandan Karfa

Department of Computer Science and Engineering



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

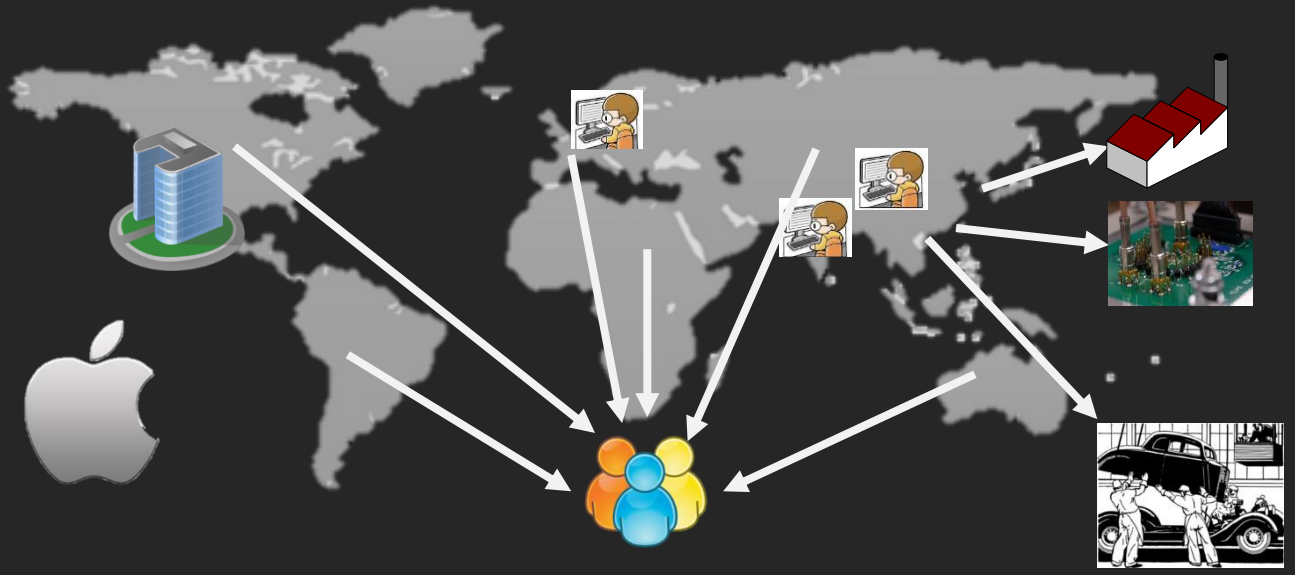
Acknowledgement

TAO: Techniques for Algorithm-Level Obfuscation during High-Level Synthesis

Christian Pilato[1], Francesco Regazzoni Ramesh Karri, Siddharth Garg

ACM/IEEE Design Automation Conference 2018
San Francisco, CA, USA, June 24-28, 2018

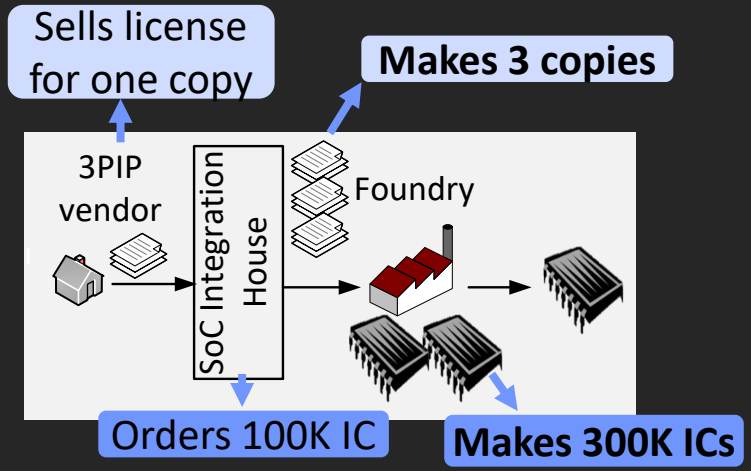
After Globalization (Today)



Design, manufacture, test & package, assembly are off-shore

Source: Digitimes and The Economist

IC/IP Piracy and Overbuilding



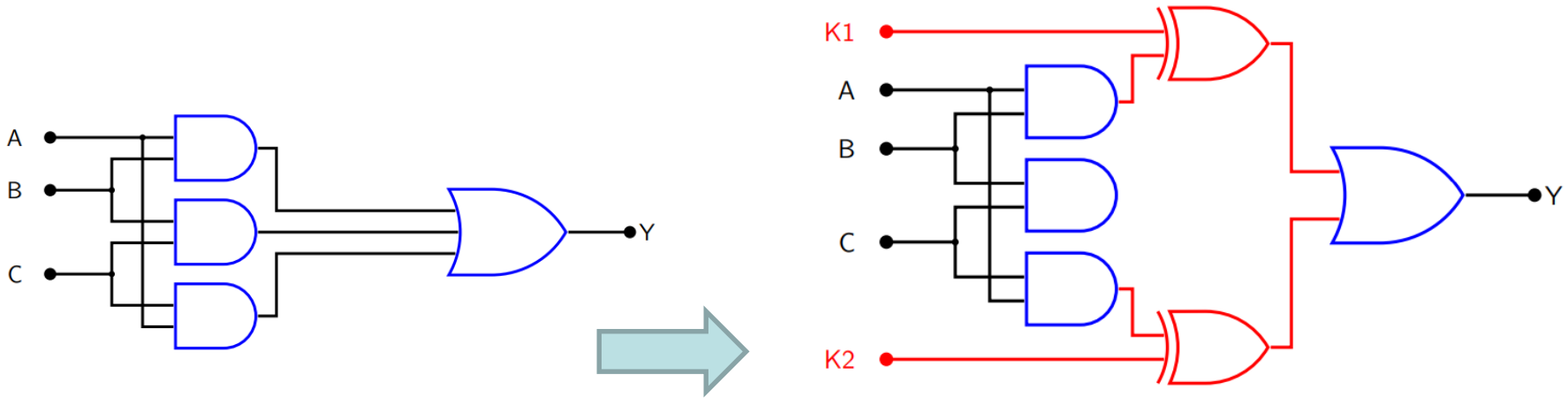




- Steal & claim ownership of an IC and/or illegal use
- Scenarios:
  - Malicious SoC integration house
  - Malicious foundry
- Real-life examples:
  - $4,000,000,000 loss per year to IC industry
  - ARM detected IP piracy in 2000

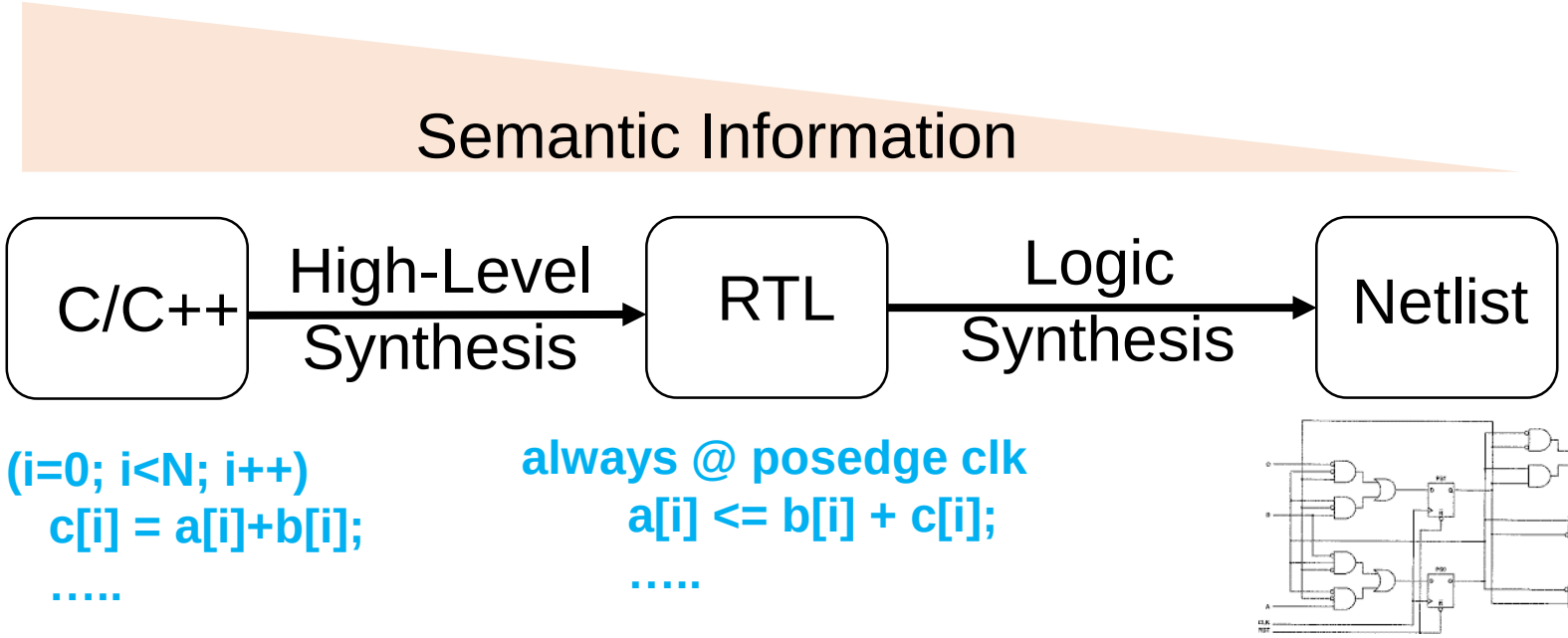
Source: SEMI and EETimes

Solution Logic Locking/Obfuscation



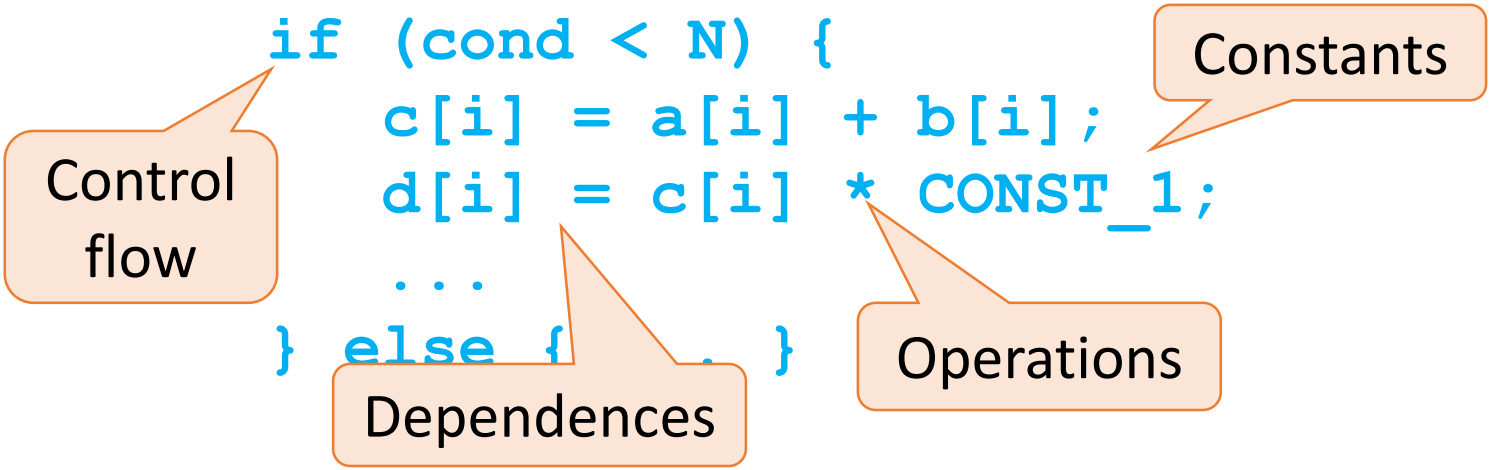
Raising the abstraction level

- Key Idea: obfuscate a design at the algorithm-level so that the obfuscation is semantically meaningful



Algorithm-level obfuscation

- An algorithm is characterized by several elements to be protected



TAO: Techniques for algorithm-level obfuscation

Technique #1: Constant obfuscation

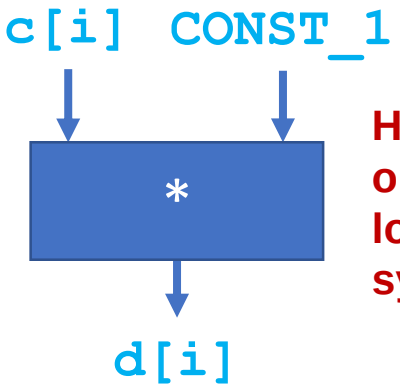
- Constants represent hard-coded values used by the algorithm (coefficients, thresholds, …)

C/C++

```
d[i] = c[i] * CONST_1;
```

Information is still present at RT Level

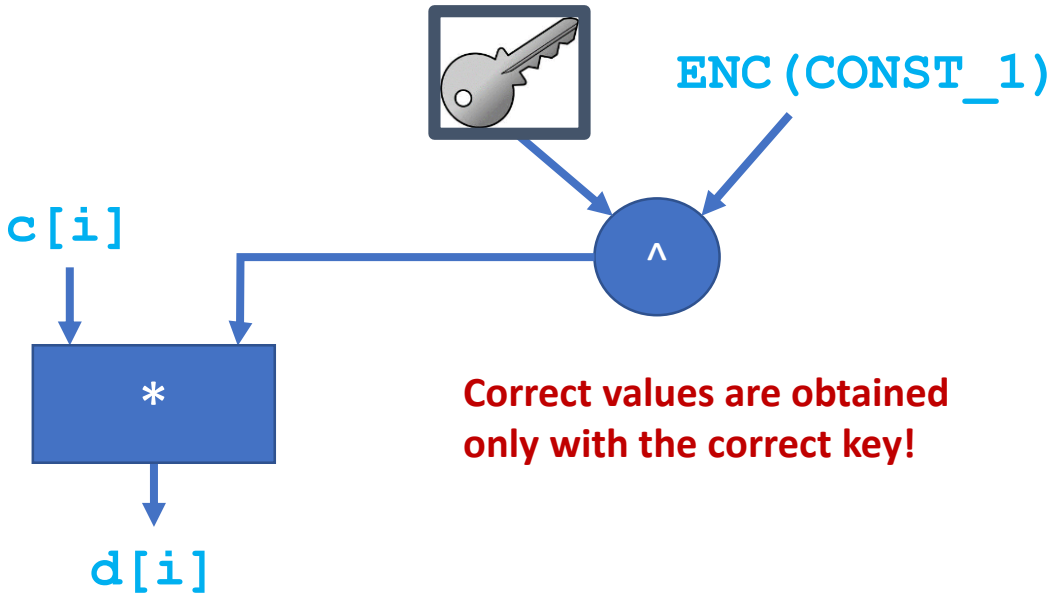
RTL



Heavily optimized by logic synthesis!

Constant obfuscation

- A *m*-bit constant is extracted and encoded using *m* working key bits



Analysis of the Technique

Obfuscated	Non-obfuscated
Data coefficients used by the algorithm	Reset values
Signal extension	Signal polarity
Mask values	

No differences concerning security, less key bits

No semantic changes

- Exact information is removed from the circuit
  - **Less information for the attacker**
  - **Less logic optimizations (area overhead)**

Technique #2: Control-flow obfuscation

- Control-flow branches describe the evolution of the algorithm

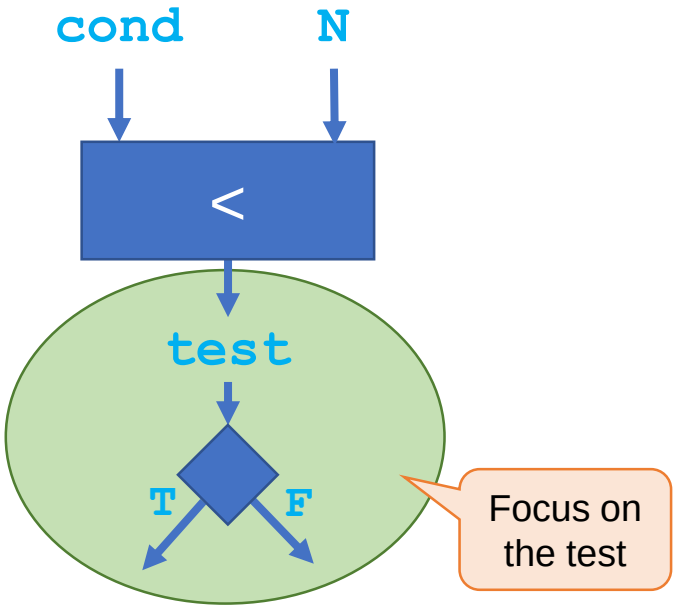
Control condition

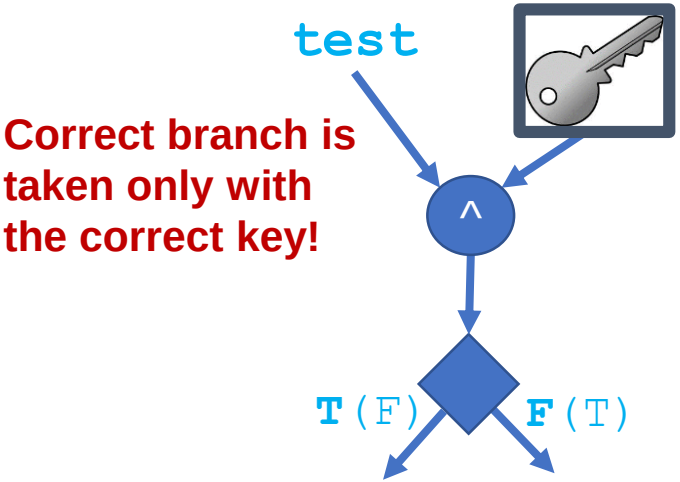
```
if (cond < N) {
 c[i] = a[i] + b[i];
   d[i] = c[i] * CONST_1;
   ...
} else { ... }
```

Attacker can get insights on the algorithm based on the branch taken

Control-flow obfuscation

- Masking of the control condition with key bit



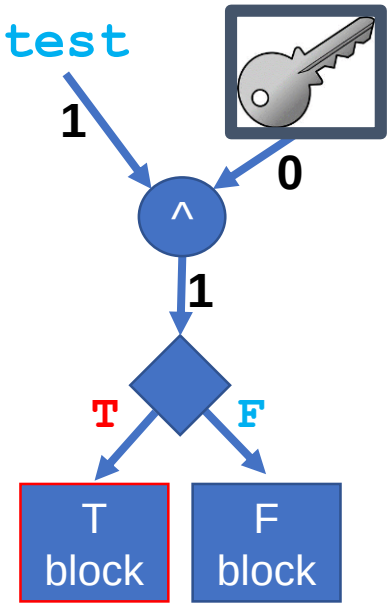


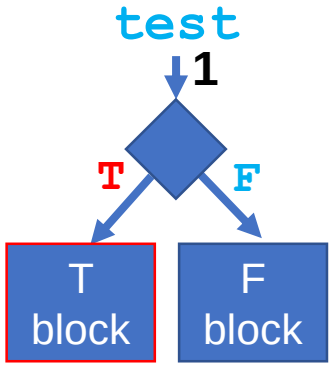
Correct branch is taken only with the correct key!

Branch reordering to ensure semantic equivalence and confuse the attacker

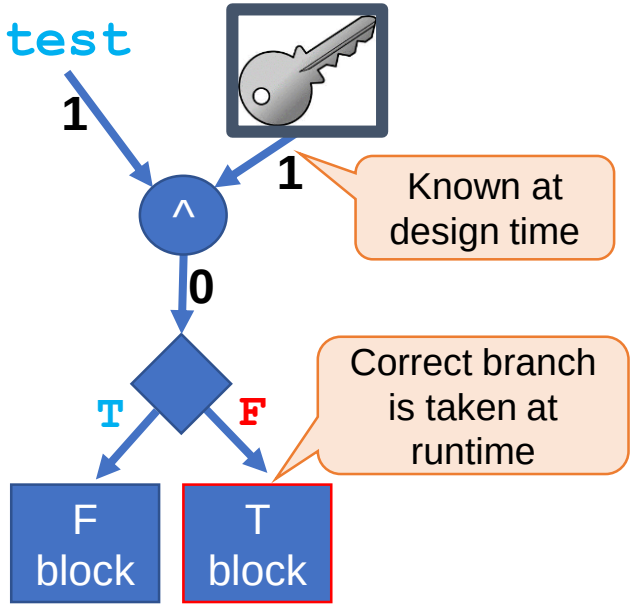
Branch reordering

- Based on the key bit for semantic equivalence





With code inspection, no intel on which is the real true/false branch without the key bit



Analysis of the Technique

- Focus on branches during data computation
  - No reset changes to allow *register inference* and prevent *synthesis issues*

- Easy to apply
  - **Minimal overhead (XOR gate)**

Almost impossible to replicate at gate level (branch info is already embedded)

Technique #3: Operation obfuscation

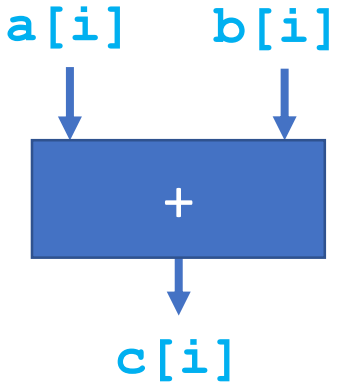
- Operations represent the computational part of the algorithm

C/C++

```
c[i] = a[i] + b[i];
```

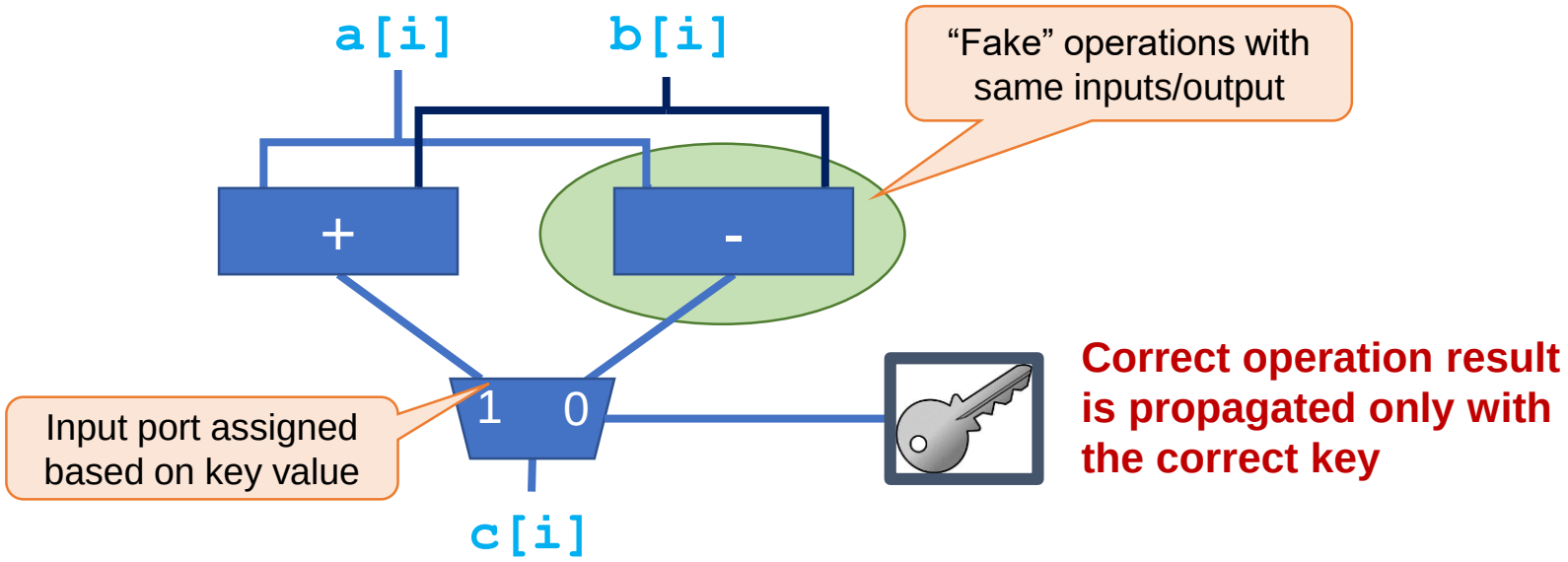
This gives intel on what the algorithm does!

RTL



Operation obfuscation

- Operator variants to camouflage the correct operation



Analysis of the Technique

- Easy to apply
  - **Reasonable area overhead (due to additional *fake* operations plus multiplexer)**

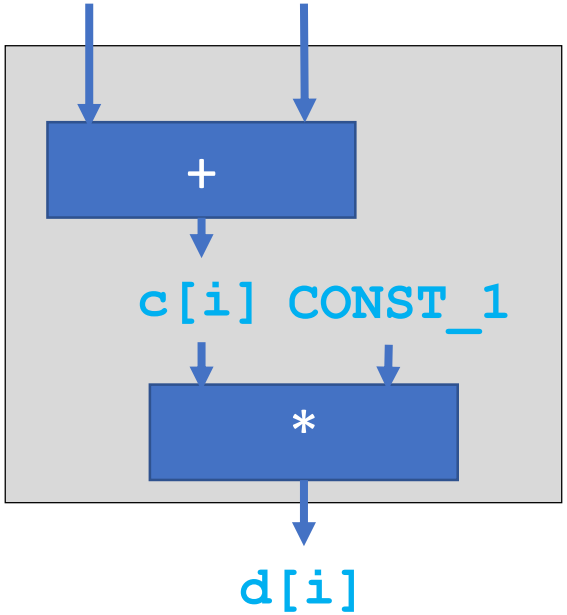
- Each operation type has a pre-defined set of alternatives to choose from

How to select operation variants is still an open issue

Technique #4: Dependence obfuscation

- Operation dependences describe how the data values are used

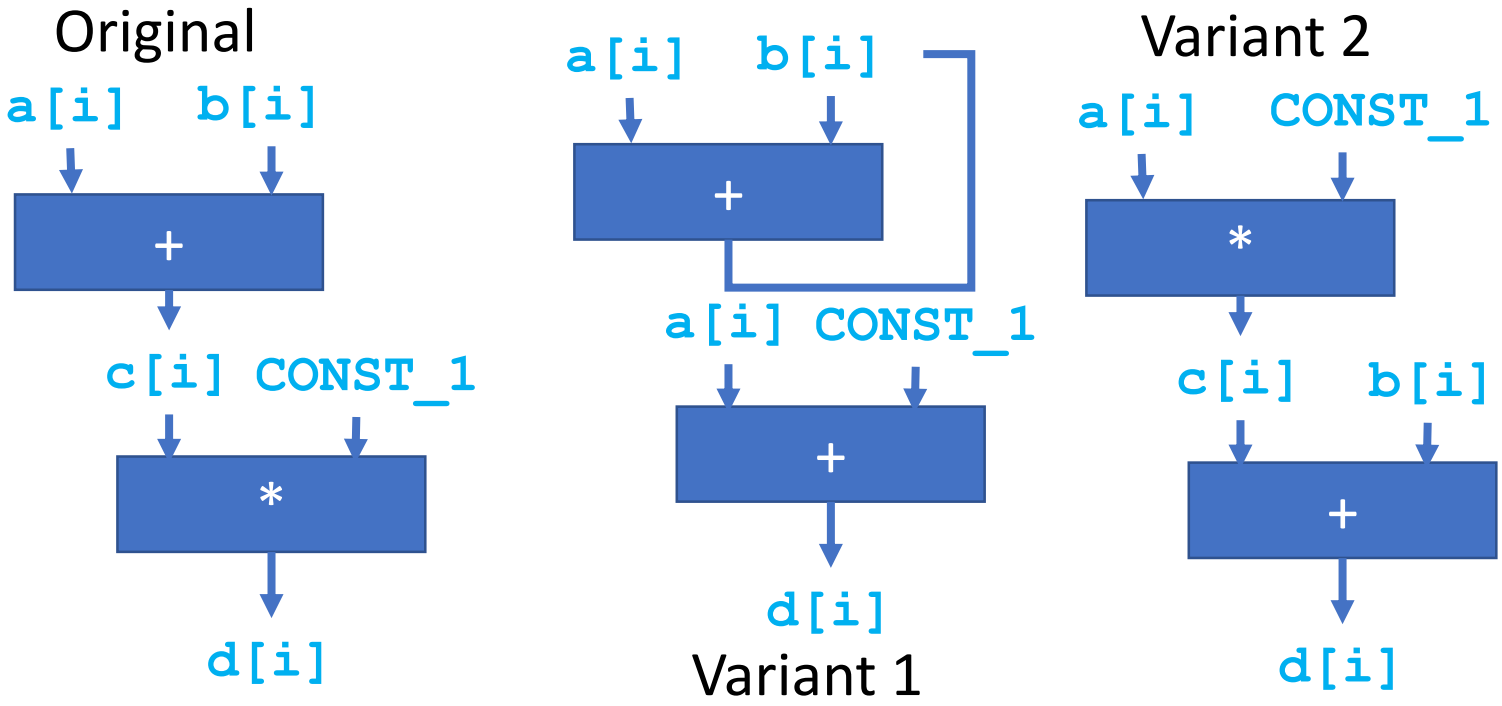
```
if (cond < N) {
    c[i] = a[i] + b[i];
    d[i] = c[i] * CONST_1;
    ...
} else { ... }
```



Very hard to compute in gate-level netlist

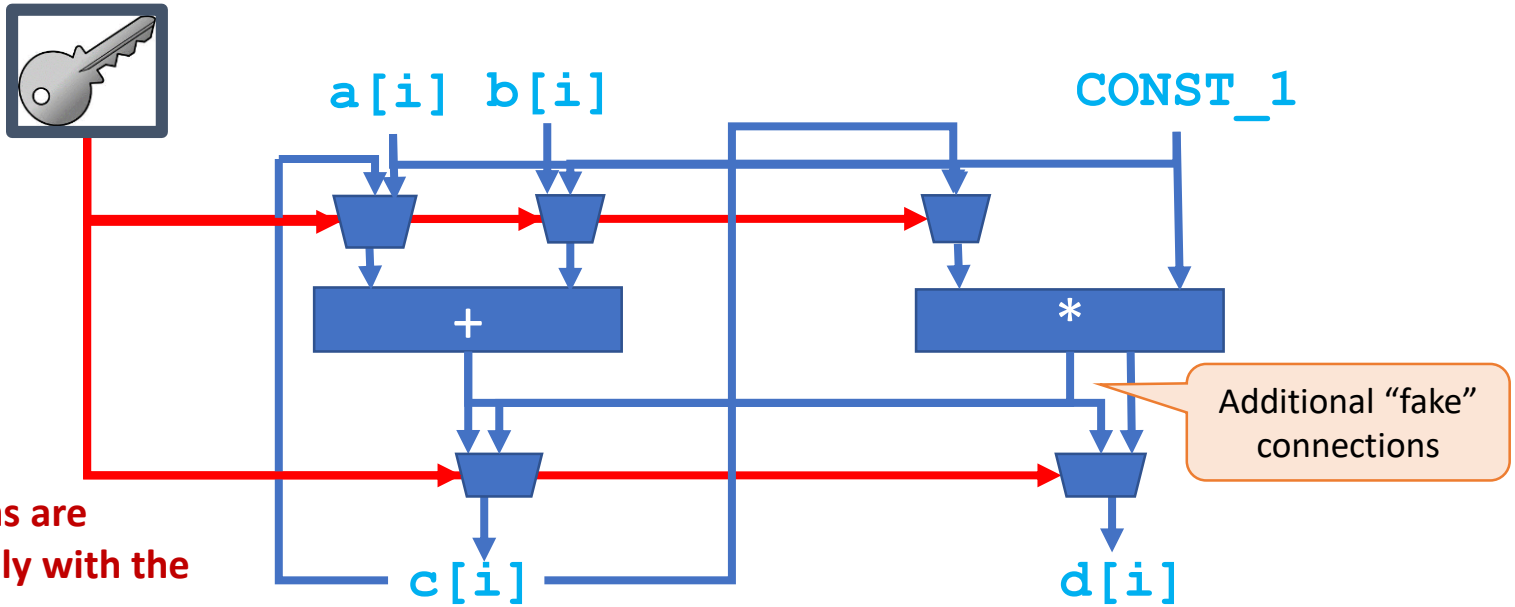
Dependence obfuscation

- *K* key bits are used to select 2^k variants



Merge and Selection of DFG Variants

- Creation and merge of DFG variants

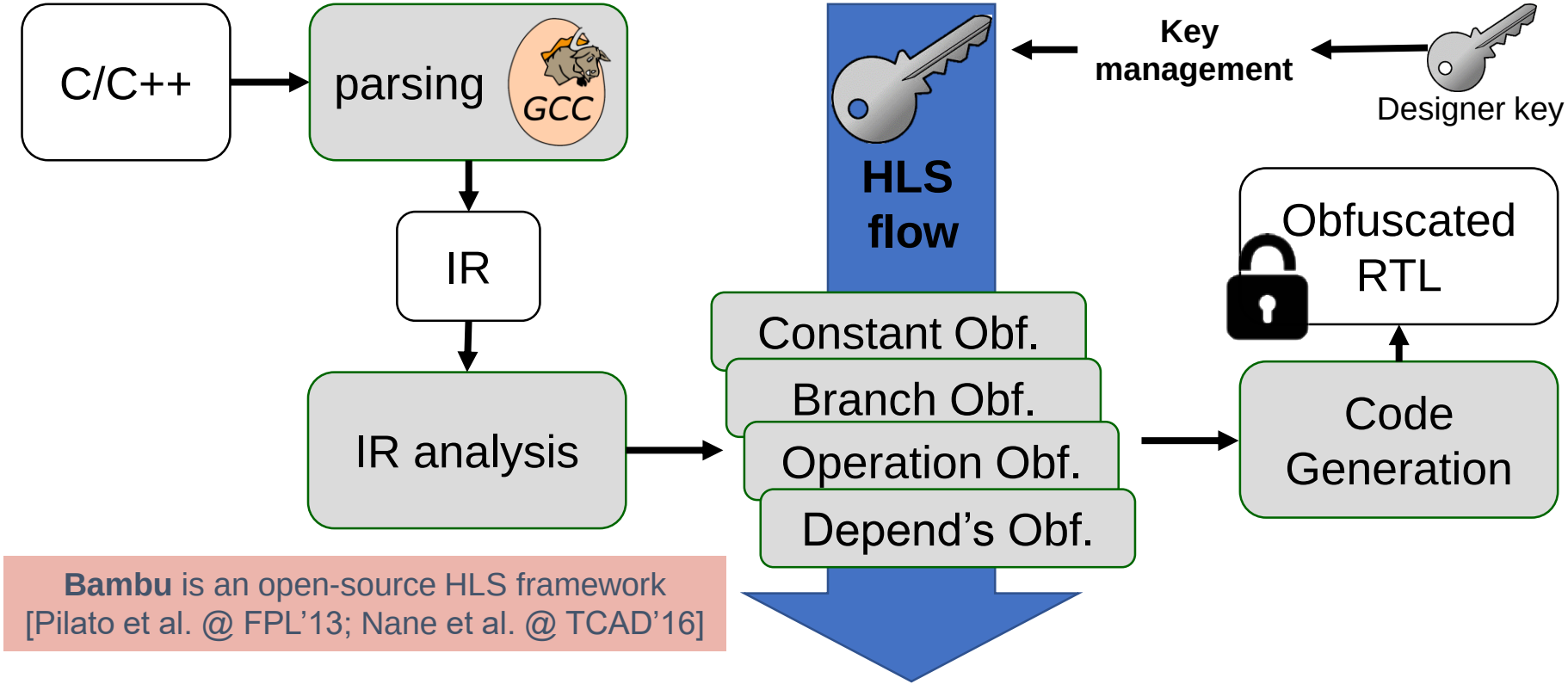


Analysis of the Technique

- Very powerful technique
  - **It creates several semantically-different but feasible code variants**
  - **It may affect component latency (different schedules)**
  - **Significant area overhead (many additional operators and connections/multiplexers)**

It can be applied only during component generation (HLS Tool-Flow)

TAO in Bambu



Bambu is an open-source HLS framework [Pilato et al. @ FPL'13; Nane et al. @ TCAD'16]

Obfuscation key bits

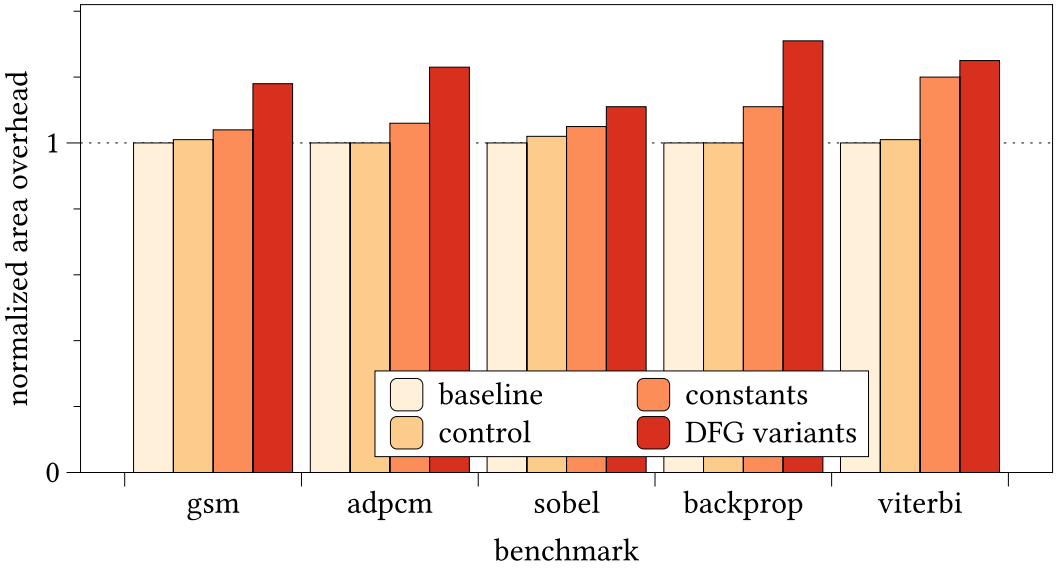
Each constant is converted into a 32-bit signal

4 bits are used for each BB (16 variants)

Design name	Const. Obf.	Branch Obf.	DFG Variants	Total key bits
GSM	4 / 128	4	88 / 352	484
ADPCM	5 / 160	5	100 / 400	565
SOBEL	2 / 64	2	11 / 44	110
BACKPROPAGATION	12 / 384	11	123 / 492	887
VITERBI	117 / 3,744	9	98 / 392	4,145

Area overhead

- Area overhead of each technique wrt the **baseline** version
  - Synopsys SAED 32nm @ 500 MHz



Conclusions and future work

- Comprehensive solution for algorithm-level obfuscation during HLS
  - Four techniques for constants, control flow, operations and dependences
  - Two solutions for key management (key folding and AES-based architecture)

- Obfuscation results validated with “output corruptability”
  - Hamming distance between output values generated with correct and incorrect keys

How to select the parts to protect to minimize the overhead?

How to ”measure” the level of obfuscation/security?

Thank you!